

基于 DoIP 的电控单元软件刷写技术要求

TheBootloaderSpecificationofDOIPECU

基于 DoIP 的电控单元软件刷写技术要求

1 范围

本文件规定了东风汽车集团有限公司车载以太网 Bootloader 需求。

本文件适用于东风汽车集团有限公司及其下属单位乘用车应用车载以太网通信的 12V 系统 ECU。适用于车载以太网 Bootloader 的开发、设计及测试等阶段。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。其中，注日期的引用文件，仅该日期对应的版本适用于本文件；不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

ISO14229-1 道路车辆-统一诊断服务（UDS）-第 1 部分：规范和要求

ISO13400-2 道路车辆-互联网协议诊断通信（DoIP）-第 2 部分：传输协议和网络层服务

ISO13400-3 道路车辆-互联网协议诊断通信（DoIP）-第 3 部分：基于 IEEE802.3 的有线车辆接口

EQC-4856UDS 诊断服务技术要求

3 术语和定义

ISO14229-1、ISO13400-2、ISO13400-3、EQC-4856 确立的以及下列术语和定义适用于本文件。本文件中的英文缩写及含义如表 1 所示。

表 1 符号与缩写

缩略语	全称
DLC	诊断数据连接器
ECU	电子控制单元
DoIPGateway	DoIP网关，具有路由和转发功能的DoIP节点
DoIPEdgeNode	边缘节点，一边终端为以太网激活线，一边终端连接外部网络的节点或主机
DoIPEntity	DoIP实体，实现DoIP协议的节点，为DoIPgateway或DoIPnode
DoIPNode	DoIP节点，实现DoIP协议，但不能执行DoIP协议路由的节点
I类ECU	嵌入式系统且支持重编程的ECU
II类ECU	基于文件系统的操作系统且支持重编程ECU
不可重编程ECU	不支持基于CAN或IP的Bootloader刷写方式的ECU，可支持供应商自定义的刷写方式
可重编程ECU	支持基于CAN或IP的Bootloader刷写方式的ECU，可支持供应商自定义的刷写方式

4 一般要求

4.1 不可重编程 ECU 的要求

为了确保重编程的执行，车载网络不可重编程的 ECU 需要满足如下要求：

- 支持诊断会话控制-扩展会话模式（10h 服务的 03h 子功能），通过此服务，ECU 可进入扩展会话模式。
- 支持诊断仪在线诊断服务（3Eh 服务的 00h/80h 子功能），通过此服务，可使 ECU 维持在非默认会话模式下。
- 支持控制 DTC 设置（85h）诊断服务，通过此服务，确保当某个 ECU 处于编程状态时，其他 ECU 不会设

置 DTC。

- d) 支持通信控制-禁止/使能发送和接收（28h）的诊断服务，通过此服务，可以降低编程过程中总线的负载。
- e) 不支持重编程相关的诊断服务，如诊断会话控制-编程会话模式（10h02h）、请求下载（34h/38h）、传输数据（36h）、请求传输退出（37h）。

除上述要求外，不可重编程 ECU 支持的全部诊断服务应遵循 ISO14229-1 要求。

4.2 可重编程 ECU 的要求

4.2.1 一般要求

支持应用软件及应用数据（包括网络配置数据和标定数据）重编程的 ECU，应当包含 Bootloader 软件。在正常运行过程中，执行的是应用软件和应用程序数据。仅当应用程序或应用程序数据无效时，或者升级时，Bootloader 软件才被激活。应用程序和应用程序数据可以同时编程或独立编程，但不允许重编程时更新 Bootloader 软件。

4.2.2 硬件要求

可重编程且正确安装的 ECU 在重编程过程中不需要从整车中移除。可重编程的 ECU 必须提供足够的存储空间保证下载，提供充足的缓存满足重编程的要求。

4.2.3 软件要求

Bootloader 软件必须存储在受保护的存储器中，确保 ECU 即使存在潜在错误，也始终是可重编程的。Bootloader 软件需包含可实现正确执行刷写流程的相应协议栈，包括 TCP/IP、DoIP 及 UDS 等。当 ECU 初始化完成且停留在 Bootloader 模式下，则 ECU 的 Bootloader 将启动一个睡眠定时器，如果收到以太网诊断报文则重启睡眠定时器，当睡眠定时器超时后，则 Bootloader 将 ECU 置为低功耗状态。推荐 I/O 端口设置成低功耗状态，且 ECU 在 Bootloader 模式下需支持网络唤醒。推荐睡眠定时器为 300s。主机厂软件版本号数据标识符 0xF189，只能通过软件刷写方式更新软件版本号。用于具有以太网的 MCU/MPU 架构的 ECU 必须支持 AB 备份。

4.2.4 安全要求

为确保下载的安全性，ECU 应满足安全需求，避免下面几种情况：

- a) 来自非法源设备的下载；
- b) 当前重编程条件不满足；
- c) 下载错误的应用程序或应用程序数据到 ECU；
- d) 软件不兼容。

4.2.4.1 安全访问

可重编程的 ECU 应支持种子和密钥的安全特性，并且可以通过安全访问服务（27h）进入访问，从而保护 ECU 免受未授权的编程动作影响。安全等级 11h/12h 用于编程会话模式。

4.2.4.2 预编程条件检查

ECU 应确保重编程的执行处于安全状态。如果预编程条件（如车速≤3km/h、整车不处于充电状态等）不满足，则应拒绝重编程请求。通过“检查预编程条件”例程控制激活 ECU 预编程条件的检验，详细信息参考第 5.5 章。

4.2.4.3 安全签名校验

ECU 需要对刷写数据进行完整性与真实性检查，即安全签名校验，具体要求及算法参考《软件刷写签名规范》。

4.2.4.4 依赖性检查

不兼容的软件不能配合使用，如果配合使用可能会导致功能异常或产生致命性错误。因此，ECU 应该验证软件兼容性来检查重编程的依赖性，包括 Bootloader 软件、应用程序与应用程序等。依赖性检查机制由 ECU 供应商制定，并经东风诊断工程师批准。当 ECU 收到“检查编程依赖性”例程控制的诊断服务请求时，ECU 将执行依赖性检查，详细的定义参考第 5.5 章。

4.2.4.5 软件验证

ECU 内部定义一个标志位，用于标识应用软件是否有效。如果重编程完整性检查和重编程依赖性检查都正确，ECU 将应用软件的标志位设置为有效。只有标志位有效时，应用软件才可以运行。

4.2.4.6 内存驱动下载（I 类控制器）

内存驱动是一种执行初始化、擦除或写入内存功能的硬件相关软件。ECU 内存必须受保护，防止执行意外擦除或重写操作。因此，ECU 需要采用软件互锁机制，即用于编程的内存驱动不应存储在 ECU 的永久性存储器中，而是下载到 ECU 的 RAM 中。下载完成后，在 ECU 返回正常操作模式之前，应从 ECU 的 RAM 缓冲区中完全删除该驱动代码。具体的内存驱动下载过程，详见第 5.3.1 章。

4.2.4.7 容错能力

Bootloader 软件存储在一个受保护的永久性存储器中，避免被意外擦除，即使没有应用软件或应用数据，ECU 依然可以支持重编程。ECU 在重编程过程中，发生以下情况时，应可以重编程：

- a) 从低/高电压状态恢复；
- b) 通信错误和超时恢复；
- c) 重启；
- d) 擦除了部分 Flash 内存。

4.2.5 源文件格式要求

ECU 需要向主机厂释放的源文件格式：

- a) I 类 ECU 使用 Intel 格式 (*.hex) 或者 Motorola 格式 (*.s19 等)。具体要求如下所示：
 - 1) 以 MotorolaS-record 格式 (*.sx,*.s19,*.mot) 和 Intel 格式 (*.hex) 发放，每个刷新文件中仅允许包含 1 个逻辑块；若每个刷写文件中包含多个逻辑块，需与诊断工程师沟通，并需采用先分段擦除再统一校验的方式。
 - 2) 刷写文件中不能有空白行与非法行，每一行的结构都要符合格式要求。
 - 3) Motorola 文件（S19 等）如果存在 S7/S8/S9,则必须在文件的最后，不能在 S1、S2 或 S3 之前。
 - 4) 刷写文件中的行顺序应遵循地址由低到高的规则进行排序。
- b) II 类 ECU 使用单个文件，使用 *.tgz、*.zip 格式。如果使用 zip 文件需要指定 zip 文件的下载绝对路径。如果使用 zip 格式，zi 文件的 ECU 内部刷写过程由 ECU 自行处理，Tester 只负责将 zip 文件通过此重编程规范正确的传输到 ECU 内部。
- c) 其他格式必须获得认可后使用。

4.2.6 通信要求

4.2.6.1 物理层

物理层需满足 EQC-4856 中定义的需求。

4.2.6.2 数据链路层

数据链路层需满足 EQC-4856 中定义的需求。

4.2.6.3 网络层参数

网络层需满足 EQC-4856、ISO13400-3 标准及本节定义的需求。

4.2.6.4 诊断层参数

程序刷新的诊断层应遵循 EQC-4856 中给定的值来设置。定时参数（如 P2Server、S3Server 等）应遵循 EQC-4856 中给定的值来设置。

4.2.7 DoIP 通信流程

DoIP 通信流程请参考 EQC-4856。

4.2.8 诊断服务要求

为了满足存储器编程的执行需求，5.5 章节定义了应用软件和 Bootloader 软件支持的诊断服务最小集。

4.3 ECU 网关要求

如果通过网关转发诊断报文，需要遵循下面规定：

- 目标地址为网关的诊断物理请求报文不需要路由；
- 目标地址为非网关的诊断物理请求报文需要路由到相应网段；
- 目标地址为网关的诊断功能寻址请求报文必须路由到各个网段。

如果对网关 ECU 进行编程，则 Bootloader 必须在其连接的网段上路由或发送诊断仪在线报文（3Eh80h）。

5 编程进程

5.1 一般信息

本章定义了将一个或多个应用软件或应用数据下载到 ECU 内存中的重编程流程。

5.2 推荐 BT 启动时序

图 1 描述了东风推荐 Bootloader 启动时序。其他时序必须获得东风诊断工程师确认，不在此规范中体现。在应用模式下，有互为备份的两个应用软件，均使用了三种不同的诊断会话模式：默认会话模式、扩展会话模式和编程会话模式。在执行安装的过程中，应用程序 A 将在 B 区执行安装，应用程序 B 将在 A 区执行安装。从诊断会话的角度，若进入编程会话模式，必须先进入扩展会话模式，意味着 ECU 不支持从默认会话模式直接跳转到编程会话模式。同样，ECU 也不支持从编程会话模式直接跳转到扩展会话模式。

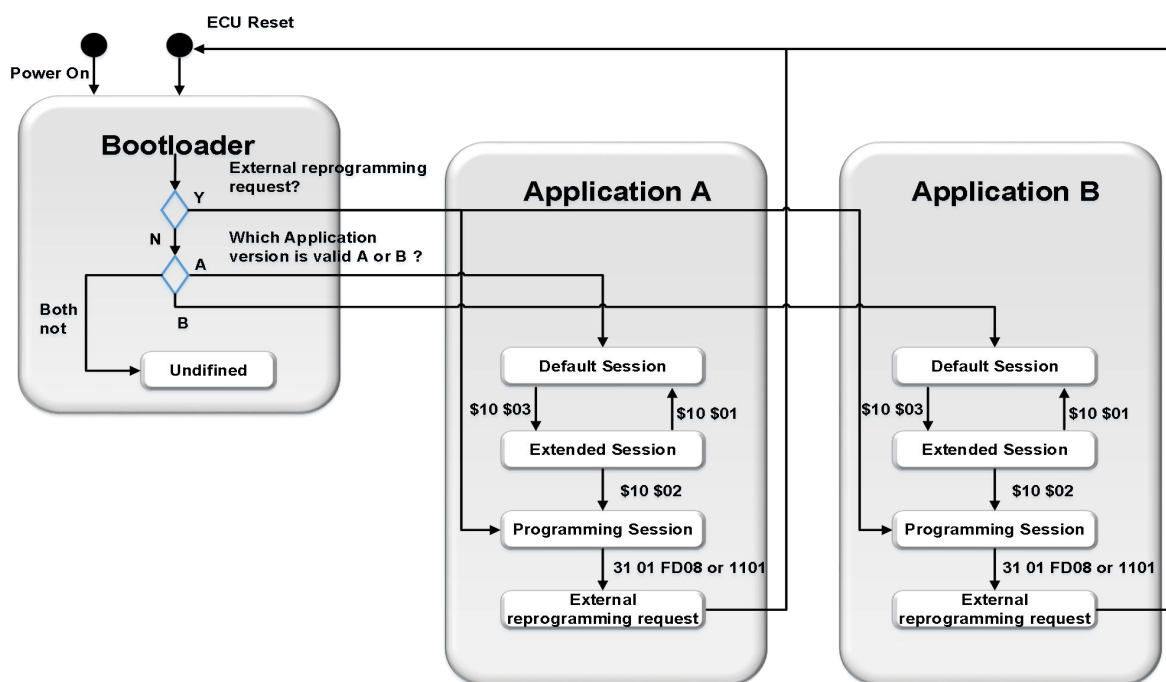


图1推荐Bootloader启动时序

上电/复位后，ECU 首先执行 Bootloader 引导代码，然后检查外部重编程请求标志位：

- 如果外部重编程请求标志位有效，则进入 A 区或者 B 区的重编程会话模式执行文件安装，将外部重编程标志位置为无效并执行安装流程，安装成功后，将更新后的分区对应的软件有效标志位置为有效，将另一分区的软件有效标志位置为无效并重启。
- 若外部重编程请求标志位无效，则检查 A 和 B 分区的应用软件标志位有效性：
 - 如果 A 区应用软件标志位有效则进入 A 区。

- 2) 如果 B 区应用软件标志位有效则进入 B 区。
- 3) 如果 A 和 B 两个分区均无效，则 ECU 停留在 Bootloader 模式下的 Undefined 会话。

5.3 I 类重编程流程

I 类重编程流程应用于嵌入式系统 ECU，分为三个编程步骤：

- a) 预编程步骤：编程前的网络准备；
- b) 主编程步骤：数据传输和软件安装；
- c) 后编程步骤：网络状态恢复。

后面章节详细描述了这三个步骤。在图 4、图 5 及图 6 的描述中，可选步骤放在了序列的右边，蓝色框和橙色框代表物理寻址，白色框代表功能寻址。如果在预编程、主编程和后编程步骤过程中，任何物理寻址的请求及响应不满足要求，则全部时序须再次执行，但最多允许重新执行一次。

5.3.1 预编程步骤

预编程步骤为 ECU 的数据下载做重编程前的网络准备工作。此步骤的请求报文采用物理寻址或功能寻址。预编程步骤如图 2 所示。

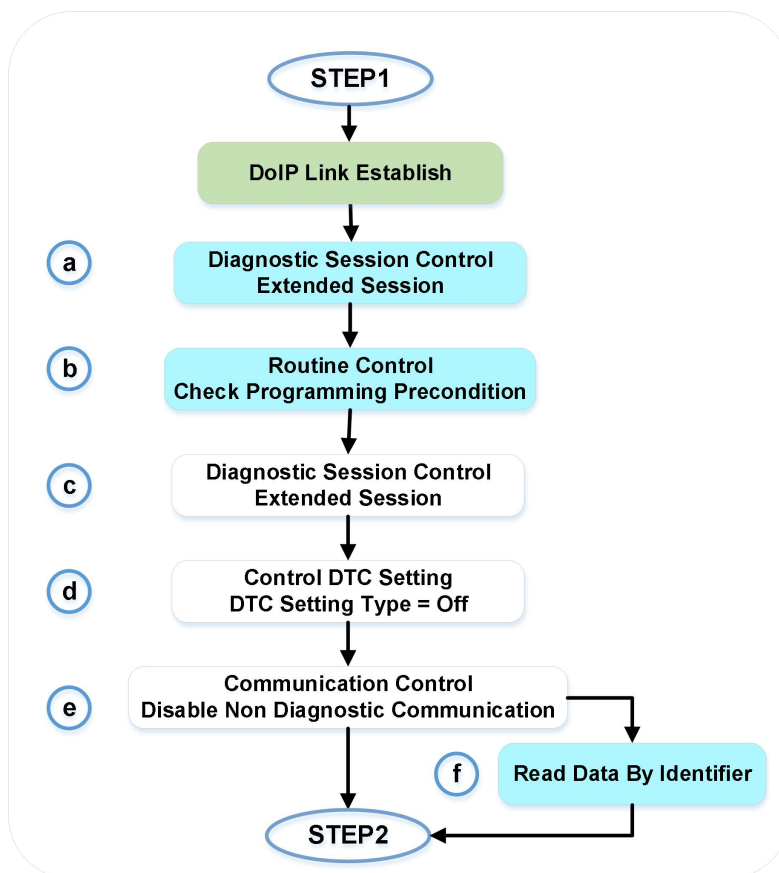


图2预编程步骤

建立 DoIP 连接：按照 DoIP 通信流程建立 DoIP 连接。

- a) 诊断会话控制 10h03h：为了实行预编程条件检查，预编程需要进入扩展会话模式，向当前待升级 ECU 发送物理寻址请求。扩展会话模式切换成功后，诊断仪应周期性（S3Client）发送诊断仪在线服务请求

(3Eh80h)，维持 ECU 的非默认会话模式（在线服务请求通过物理寻址发送给当前诊断 ECU）。

- b) 例程控制“检查预编程条件 31h01h02h03h：用于检查 ECU 编程条件，确保系统安全，预编程检验条件由 ECU 决定，如果有任何不安全的因素，ECU 应该拒绝编程。
- c) 诊断会话控制 10h83h：为了控制 DTC 设置，预编程需要进入扩展会话模式，向所有 ECU 发送功能寻址请求。扩展会话模式切换成功后，诊断仪应周期性（S3Client）发送诊断仪在线服务请求（3Eh80h），维持 ECU 的非默认会话模式（在线服务请求通过功能寻址发送给所有 ECU）。
- d) 控制 DTC 设置 85h82h：诊断仪发送 DTC 设置类型为“关闭”的控制 DTC 设置服务请求。此请求通过一帧功能寻址请求报文发送给所有 ECU。
- e) 通信控制 28h83h03h：诊断仪通过通信控制（28h）服务请求，禁止非诊断报文的发送和接收。请求的控制类型参数置为“禁止接收和发送”，通信类型置为“网络管理报文和常规应用报文”。此请求通过一帧功能寻址请求报文发送给所有 ECU。
- f) 读取数据 22hxxhyyh（可选）：禁止正常通信后，读取 ECU 的编程状态（如编程的应用软件和数据）。从被编程的 ECU 读取服务器标识数据，如应用软件标识、应用数据标识、引导软件标识。此步骤为可选项，读取内容由 ECU 供应商定义。

5.3.2 编程步骤

在预编程步骤后，进行主编程步骤。主编程时序是对单个 ECU 编程事件的应用，因此所有服务的请求都使用物理寻址。图 3 描述了在主编程步骤中执行的各个服务。

- a) 诊断会话控制 10h02h：收到一个寻址方式为物理寻址，子功能为编程会话模式的诊断会话控制（10h）服务后，ECU 启动 Bootloader。ECU 首先发送肯定响应，再执行编程会话模式的跳转动作，跳转完成后应保持诊断连接。
- b) 安全访问 27h11h/12h：下载前，编程事件必须通过安全访问，确保只有合法的诊断仪才能对 ECU 进行下载操作。
- c) 读当前运行分区信息 22hF0hF0h：对支持 AB 备份双分区运行的 ECU，必须支持此步。不支持双分区运行的 ECU 不需要支持。Tester 根据读取到的当前运行分区信息，选择对应的版本文件进行刷写。如果 ECU 返回 A，Tester 将选择 B 版本文件进行刷写，如果 ECU 返回 B，Tester 将选择 A 版本文件进行刷写。如果 ECU 不支持此步骤，返回 NRC31 表示不支持此 DID，Tester 取通用版本文件进行刷写，否则应该取对应的 A 或者 B 区的版本文件进行刷写。
- d) 写入数据 2EhF1h84h：擦除内存例程前，强制将“指纹”数据写入 ECU 内存中。“指纹”标识了哪个诊断仪对 ECU 内存做了修改。下载前，诊断仪将首先写入“指纹”，程序下载完成后，ECU 将其存储。
- e) 驱动下载 34h, 36h, 37h, 31h：当 ECU 的非易失性存储单元中没有存储内存驱动时，将执行内存驱动的下载。下载应该按照如下时序进行：请求下载、传输数据、请求传输退出。下载完所有字节后，用“安全签名校验”例程（31h01h02h02h）检查所有的字节是否正确传输。
- f) 例程控制-“擦除内存”31h01hFFh00h：为了应用软件和数据下载，ECU 的内存将被擦除。此步骤通过例程控制服务（31h）执行擦除内存。如果调用擦除内存例程，那么应用软件的标志位将置为无效。
- g) 读取下载过程 34h, 36h, 37h：应用软件或数据的每一个连续数据块（也称为段，可作为一个完整的应用软件或数据，也可能是应用软件或数据的一部分）下载到 ECU 非易失性内存中，都是遵循下面的服务顺序完成数据传输：
 - 1) 请求下载（34h）；
 - 2) 传输数据（36h），传输过程中 Tester 可能会重传 36 消息，ECU 需要根据 ISO14229 的要求正确处理这种场景的消息传输。为了防止 36 消息无限循环，建议 ECU 内部对 BlockSequenceCounter 重置次数计数，达到一定次数后，返回 NRC73 的否定响应。
 - 3) 请求传输退出（37h）。单个应用软件或数据块可能需要多个数据传输（36h）请求报文完成传输（当数据块长度超出网络层缓存大小时，就会出现这种情况）。
- h) 例程控制-“安全签名校验”31h01h02h02h：用来检查逻辑块的完整性与真实性。
- i) 例程控制-“阻止自动切区”31h01hDDh0Fh：此步骤用于限制支持 A/B 区 ECU 做自动切区激活的动作，

支持 A/B 区运行的部件必须支持此步骤（如无法支持需要向 OEM 申请认可），或非 A/B 区运行部件可不支持此步骤。支持 A/B 区的传统 ECU 在刷写程序完成传输后可下发该指令阻止自动切区的动作，防止发生非预期的自动切区动作。此例程为可选步骤，该例程可提供给远程并行升级场景使用，近端升级场景下可以选择不支持。

一旦下发启动阻止自动切区指令，ECU 需要在本次上电周期内的下一次包含自动切区激活动作的步骤（如依赖性检查步骤或者复位步骤）中禁止自动更新启动分区标识，在完成禁止自动更新启动分区标识后，阻止自动切区即须失效。在运行分区切换流程中（参考 5.5.2 章节），ECU 应该强制立即做 A/B 区切换并忽略阻止自动切区命令。本规范附录 B 提供了阻止自动切区的实现方案供参考。

- j) 例程控制-“检查编程依赖性”31h01hFFh01h: 一旦完成所有应用软件或数据的下载，诊断仪将启动一个例程，触发检查重编程的依赖性。由 ECU 供应商定义检查内容，但必须确保所有逻辑块的兼容性和一致性。当依赖性检查通过后，将应用软件的标志位置为有效。

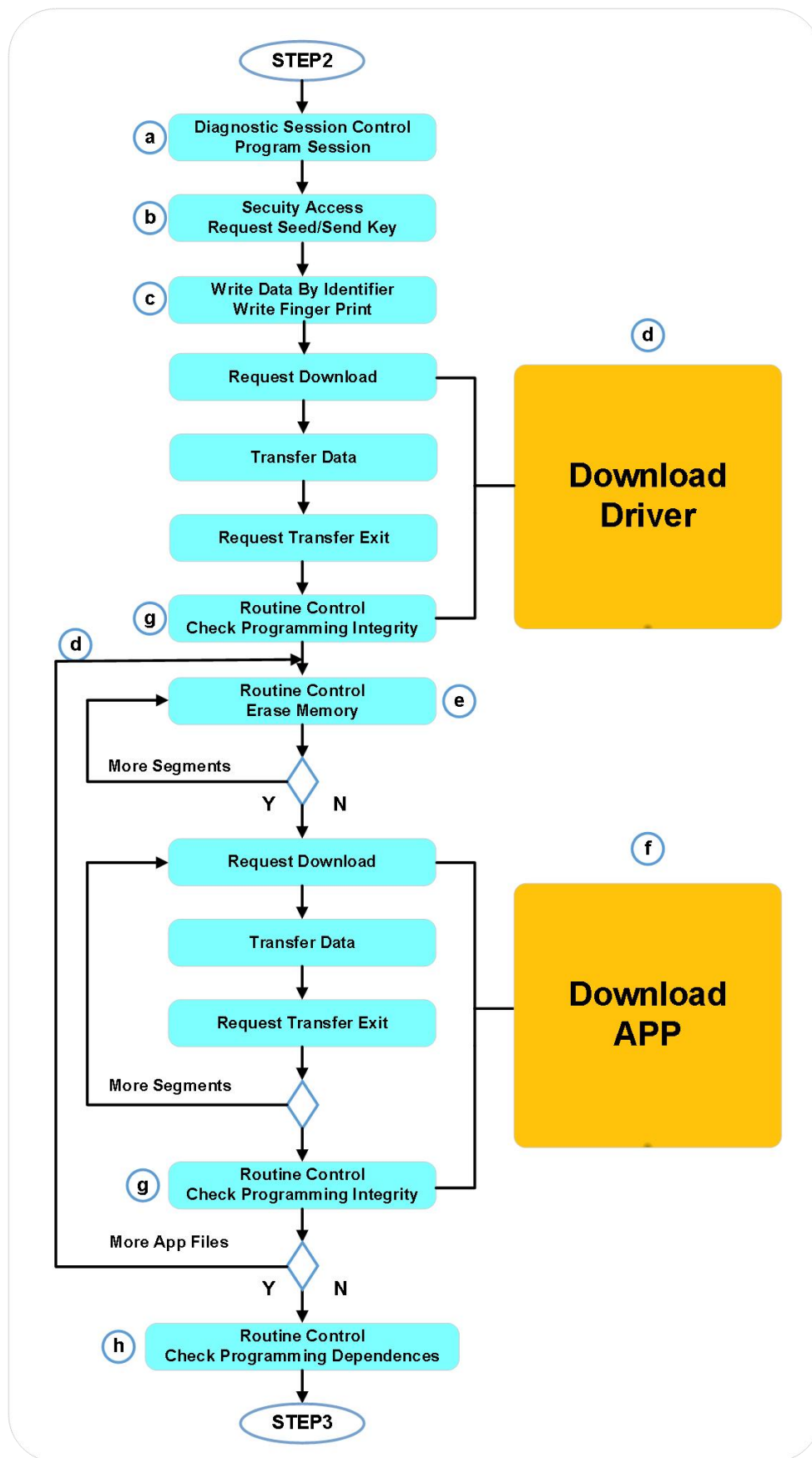


图3主编程步骤

5.3.3 后编程步骤

后编程步骤如下图 4 所示：

- a) ECU 复位 11h01h: 诊断仪通过物理寻址向待刷写 ECU 发送一个复位类型为硬复位(01h)的 ECU 复位(11h)

服务请求。

ECU 复位服务请求使 ECU 结束重编程过程，返回正常的操作模式。内存驱动代码必须从 RAM 缓存中完全清除，避免意外激活后，进行非预期的内存擦除或程序操作。

建立 DoIP 连接：诊断仪收到复位正响应后，按照 DoIP 通信流程尝试建立 DoIP 连接。

- b) 诊断会话控制 10h81h：诊断仪发送一个默认会话模式的功能寻址请求报文，诊断会话控制（10h）的接收使所有服务器启动默认会话，该请求发送到编程事件中包含的所有 ECU 或由于编程事件而进入非默认会话模式的 ECU，使 ECU 跳到默认会话模式。

注：进入默认会话模式后，通信控制（28h）服务和控制 DTC 设置（85h）服务应复位，即被禁用的正常通信在会话切换到默认会话时应重新恢复。

- c) 清除诊断信息 14hFFhFFhFFh：如果重编程 ECU 在编程步骤中重启，当重编程 ECU 在默认会话模式运行时，网络上其它的 ECU 仍不能在正常通信状态工作，此时，一些重编程 ECU 就应该通过功能寻址的清除诊断信息（14h）服务清除 DTC。

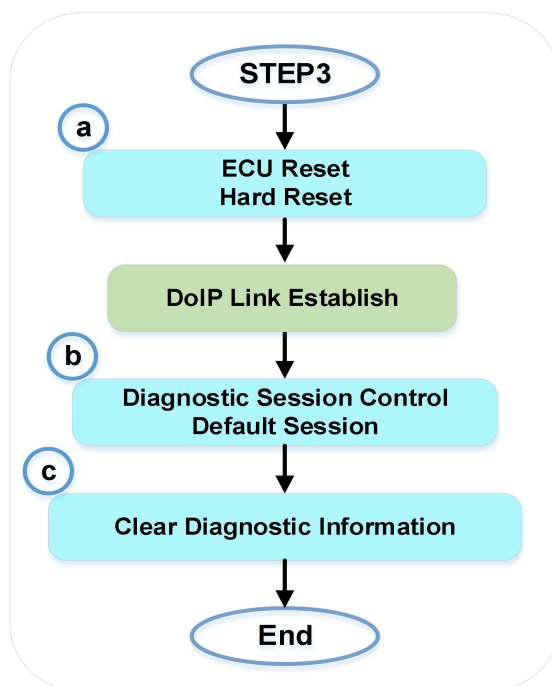


图4后编程步骤

5.4 II类重编程流程

II类重编程时序应用于带有操作系统的 ECU，分为三个编程步骤：

- a) 预编程步骤：编程前的网络准备；
- b) 主编程步骤：数据传输和软件安装；
- c) 后编程步骤：网络状态恢复。

后面章节详细描述了这三个步骤。如果在预编程、主编程和后编程步骤过程中，任何物理寻址的请求及响应不满足要求，则全部时序须被再次执行，但诊断仪最多允许重新执行一次。

5.4.1 预编程步骤

预编程步骤为 ECU 的数据下载做重编程前的网络准备工作。此步骤的请求报文采用物理寻址或功能寻址。预编程步骤如图 5 所示。按照 DoIP 通信流程建立 DoIP 连接：

- a) 诊断会话控制 10h01h：首先需要发送\$1001 的物理寻址消息，请求使目的 ECU 进入默认会话模式。
- b) 诊断会话控制 10h83h：为了实行预编程条件检查，预编程需要进入扩展会话模式，向所有 ECU 发送\$1083 的功能寻址消息，请求使网络中所有 ECU 进入扩展会话模式。Tester 发送完此报文后，需要至少延时 1 秒再执行后续步骤。
- c) 例程控制“检查预编程条件” 31h01h02h03h：用于检查 ECU 编程条件，确保系统安全，预编程检验条件由

ECU 决定，如果有任何不安全的因素，ECU 应该拒绝编程。

- d) 控制 DTC 设置 85h82h: 诊断仪发送 DTC 设置类型为“关闭”的控制 DTC 设置服务请求。此请求通过一帧功能寻址请求报文发送给所有 ECU。
- e) 通信控制 28h83h03h: 诊断仪通过通信控制（28h）服务请求，禁止非诊断报文的发送和接收。请求的控制类型参数置为“禁止接收和发送”，通信类型置为“网络管理报文和常规应用报文”。此请求通过一帧功能寻址请求报文发送给所有 ECU。由于此服务禁止了网络管理报文，ECU 需要支持诊断报文作为唤醒源，防止 ECU 在刷写过程中进入休眠。
- f) 读取数据 22hxxhyyh: 禁止正常通信后，读取 ECU 的编程状态（如编程的应用软件和数据）。从被编程的 ECU 读取服务器标识数据，如应用软件标识、应用数据标识、引导软件标识。外部诊断设备 Tester 应该读取这些信息或者更多的信息，此步骤 ECU 的实现情况不应该影响整个刷写流程。ECU 和 Tester 需要保证此步骤的执行结果（ECU 否定响应，ECU 超时未响应或者 Tester 没有执行此步骤等）不应该影响后续刷写流程的执行。

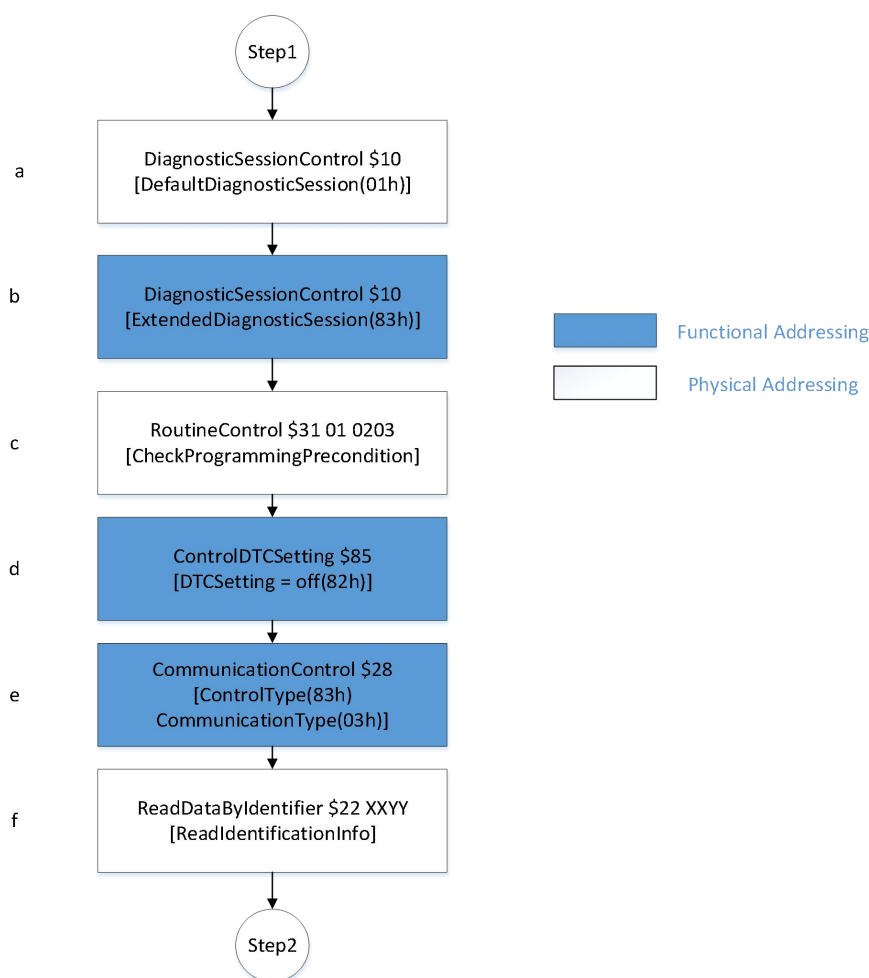


图5预编程步骤

5.4.2 编程步骤

在预编程步骤后，进行主编程步骤。主编程时序是对单个 ECU 编程事件的应用，因此所有服务的请求都使用物理寻址。图 6 描述了在主编程步骤中执行的各个服务。

- a) 诊断会话控制 10h02h: 收到一个寻址方式为物理寻址，子功能为编程会话模式的诊断会话控制（10h）服务后。ECU 首先发送肯定响应，再执行编程会话模式的跳转动作。
- b) 建立 DoIP 连接: 此步骤为可选操作，如果 ECU 复位会导致自己的 DoIP 断链，Tester 需要做链路检测和重连。Tester 先延时 1 秒，然后周期性发送组播的车辆发现请求消息（DoIPVehicleDiscoveryRequest 消息）来确认目标 ECU 与 Tester 的 DoIP 通道是否已经就绪，ECU 收到车辆发现请求后给出应答。Tester 以 500ms

的周期发送车辆发现请求，收到目标 ECU 连续 2 次正确响应后，Tester 尝试恢复与目标 ECU 的 DoIP 链路，默认连接超时 T1=10 秒。当目标 ECU 连续 2 次正确响应了车辆发现消息或 T1 定时器超时后，Tester 停止发送车辆发现消息。

- c) 安全访问 27h11h/12h: 下载前，编程事件必须通过安全访问，确保只有合法的诊断仪才能对 ECU 进行下载操作。
- d) 读当前运行分区信息 22hF0hF0h: 对支持 AB 备份双分区运行的 ECU，必须支持此步。不支持双分区运行的 ECU 不需要支持。Tester 根据读取到的当前运行分区信息，选择对应的版本文件进行刷写。如果 ECU 返回 A，Tester 将选择 B 版本文件进行刷写，如果 ECU 返回 B，Tester 将选择 A 版本文件进行刷写。如果 ECU 不支持此步骤，返回 NRC31 表示不支持此 DID，Tester 取通用版本文件进行刷写，否则应该取对应的 A 或者 B 区的版本文件进行刷写。
- e) 写入数据 2EhF1h84h: 在数据传输前，强制将“指纹”写到 ECU 内存中。“指纹”标识了哪个诊断仪什么时间对 ECU 内存做了修改。ECU 需要维护一个尝试编程次数的计数器 (DID=0xF0F1)，在正响应返回之前，此计数器加 1，此计数值应该持久保存。如果编程次数超过 ECU 最大尝试编程次数（设置为 0 时，不限制尝试编程次数，默认为 0），ECU 应该返回 NRC22。
- f) 下载过程 38h, 36h, 37h: 将每一个文件下载到 ECU 非易失性内存中，智能部件文件下载，不需要下载驱动文件，直接使用 SID=0x38 进行文件传输下载。如果智能部件有驱动文件则需要参考 5.5.1 章节进行文件的传输下载。对于采用\$38 进行下载的部件，建议把所有需要下载的文件，打包成一个 zip 文件后进行下载。FileDownloadStep 的下载流程图见如下图 6 所示：

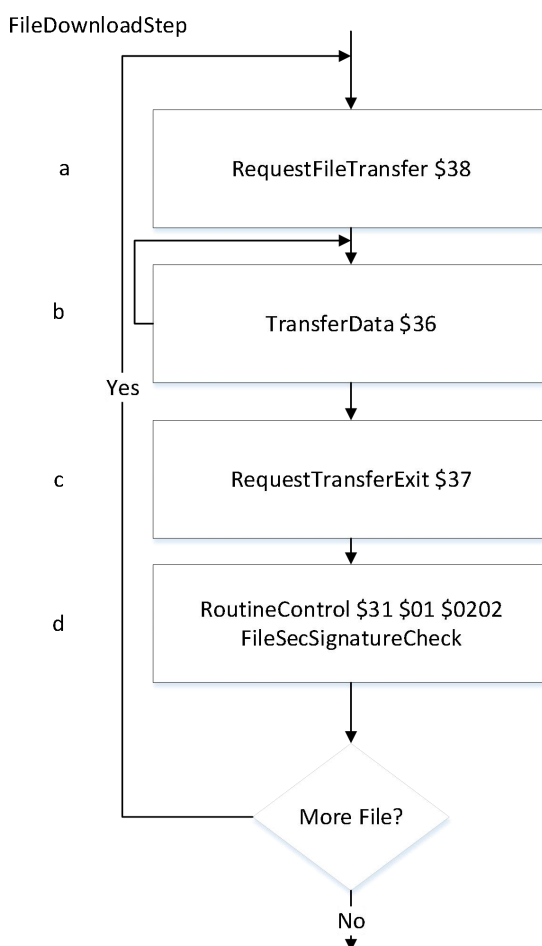


图 6 智能部件文件下载流程

- 1) 请求文件传输（38h），智能部件一般使用 SID=0x38 进行文件的传输，不需要进行驱动文件下载。
- 2) 传输数据（36h），Tester 使用此服务将编程的数据传输到 ECU。参数 BlockSequenceCounter 用作块计数器，以确保 Blocks 正确的被传输。BlockSequenceCounter 从 1 开始，依次累加，当累加到 FFh 时，下次从 0 开始。传输过程中 Tester 可能会重传 36 消息，ECU 需要根据 ISO14229 的要求正确处理

这种场景的消息传输。为了防止\$36 消息无限循环，建议 ECU 内部对 BlockSequenceCounter 重置次数计数，达到一定次数后，返回 NRC73 的否定响应。

3) 请求传输退出 (37h)。

单个文件可能需要多个数据传输 (36h) 请求报文完成传输 (当数据块长度超出网络层缓存大小时，就会出现这种情况)。

- g) 例程控制-“安全签名校验” 31h01h02h02h: 当一个文件中的所有软件段全部传输完毕后，通过例程控制触发 ECU 内部进行安全签名的校验。如果 ECU 验证安全签名成功，则此例程返回成功，ECU 将给出验证成功的肯定响应消息 (驱动文件需要激活驱动)。否则，将发送安全签名校验失败的肯定响应消息。当 Tester 收到此消息的校验失败的肯定响应消息或者 NRC 否定响应时，Tester 将终止重编程流程。软件包的安全签名算法选择，参考《软件刷写签名规范》。此消息是否携带安全签名值，跟具体选择的安全签名方案相关，各项目所采用安全签名方案请参考《软件刷写签名规范》，并与主机厂诊断工程师确认。
- h) 例程控制-“阻止自动切区” 31h01hDDh0Fh: 此步骤用于限制支持 A/B 区 ECU 做自动切区激活的动作，支持 A/B 区运行的部件必须支持此步骤 (如无法支持需要向 OEM 申请认可)，或非 A/B 区运行部件可不支持此步骤。支持 A/B 区的传统 ECU 在刷写程序完成传输后可下发该指令阻止自动切区的动作，防止发生非预期的自动切区动作。此例程为可选步骤，该例程可提供给远程并行升级场景使用，近端升级场景下可以选择不支持。

一旦下发启动阻止自动切区指令，ECU 需要在本次上电周期内的下一次包含自动切区激活动作的步骤 (如依赖性检查步骤或者复位步骤) 中禁止自动更新启动分区标识，在完成禁止自动更新启动分区标识后，阻止自动切区即须失效。在运行分区切换流程中 (参考 5.5.2 章节)，ECU 应该强制立即做 A/B 区切换并忽略阻止自动切区命令。本规范附录 B 提供了阻止自动切区的实现方案供参考。

- i) 例程控制-“检查编程依赖性” 31h01hFFh01h: 一旦完成所有应用软件或数据的下载，诊断仪将启动一个例程，触发检查重编程的依赖性。由 ECU 供应商定义检查内容，但必须确保所有逻辑块的兼容性和一致性。当依赖性检查通过后，将应用软件的标志位置为有效。

ECU 内部维护一个编程依赖检查成功的计数器 (DID= 0xF0F3)，并保存在 Flash 中，每次编程依赖 检查成功后此计数器加 1，当达到最大允许编程次数 (最大允许编程次数设置为 0 时，不限制编程次数，默认为 0)，ECU 应该禁止编程，编程依赖检查返回 NRC22 的否定响应消息。

- j) 例程控制-“启动安装” 31h01hFDh08h: 当所有数据传输完毕且依赖性检查通过后，诊断仪会通过“启动安装”例程触发 ECU 对升级文件安装，若安装前 ECU 需要复位，先执行复位再安装升级文件，收到此请求后应将应用软件有效位置为无效。
- k) 获取安装进度和结果 22hF0hF2h: Tester 应该以 1 秒的周期调用此服务，获取安装进度和结果，智能部件必须支持此步骤。ECU 超时未响应此消息，Tester 不能当做错误处理，应该继续查询，直到获取到成功并且进度 100%或者 ECU 响应安装或校验失败。此过程中 Tester 如果检测 DoIP 链路断开，Tester 应该做链路重连，直到获取安装成功或者安装失败。ECU 可能由于故障原因一直无法完成安装，此步骤应该设置超时时间，超时时间 T2=30 分钟没有完成或者结束，Tester 应该终止本次重编程流程。在编程序列的安装进度获取中，FileInstallStatus 字段应该只允许响应 00h/01h/02h/04h，不允许响应 03h/05h。

ECU 启动安装过程中需断开 TCP 连接，无法建立诊断 TCP 连接，直至升级完成并重启。在此过程中诊断仪会周期尝试性重新建立 TCP 连接。

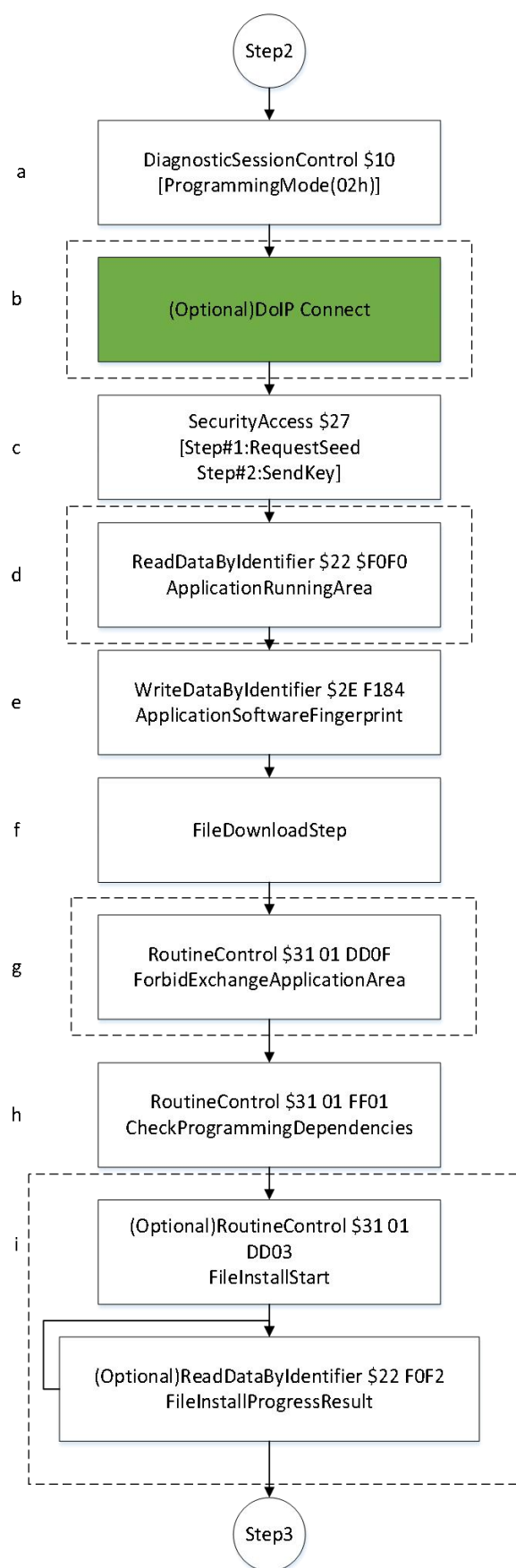


图7 主编程步骤

5. 4. 3后编程步骤

- a) 通讯控制 28h80h03h: 发送此服务恢复所有 ECU 的网络管理报文和通用报文的发送和接收。
- b) 复位 11h01h: 发送此服务将使 ECU 重启, 并生效下载的应用软件/数据。ECU 在收到先回复响应, 再主动关闭和清理 DoIP 链路, 然后再执行复位。Tester 在发送完此命令, 收到正响应后, 需要等待 5 秒, 再执行后续步骤。
- c) 建立 DoIP 连接: 诊断仪 ECU 建立 TCP 连接后, 按照 DoIP 通信流程尝试建立 DoIP 连接。
- d) 获取安装后校验结果 22hF0hF2h: 智能部件必须支持此步骤。智能部件完成版本安装并重启后, 运行在新版本后可能需要查询内部安装和校验的信息才能确认本次升级是否成功, 如 ECU 的私有部件的升级结果, 失败原因, 故障码等, 在后程序序列的激活进度获取中, FileActiveStatus 字段应该只允许响应 00h/01h/03h/05h, 不允许响应 02h/04h。
- e) 诊断会话控制 10h03h: 发送\$1003 的物理寻址消息, 请求目的 ECU 进入扩展会话模式。
- f) 清除诊断信息 14hFFhFFhFFh: Tester 使用物理寻址清除目的 ECU 的故障码。
- g) 控制 DTC 设置 85h81h: Tester 使用此服务使能所有 ECU 的 DTC 设置, 启动所有 ECU 的设置 DTC 功能。
- h) 诊断会话控制 10h81h: Tester 将发送此消息使所有 ECU 启动默认会话模式。回到默认会话后, ECU 也会自动打开通讯, 使能故障码。

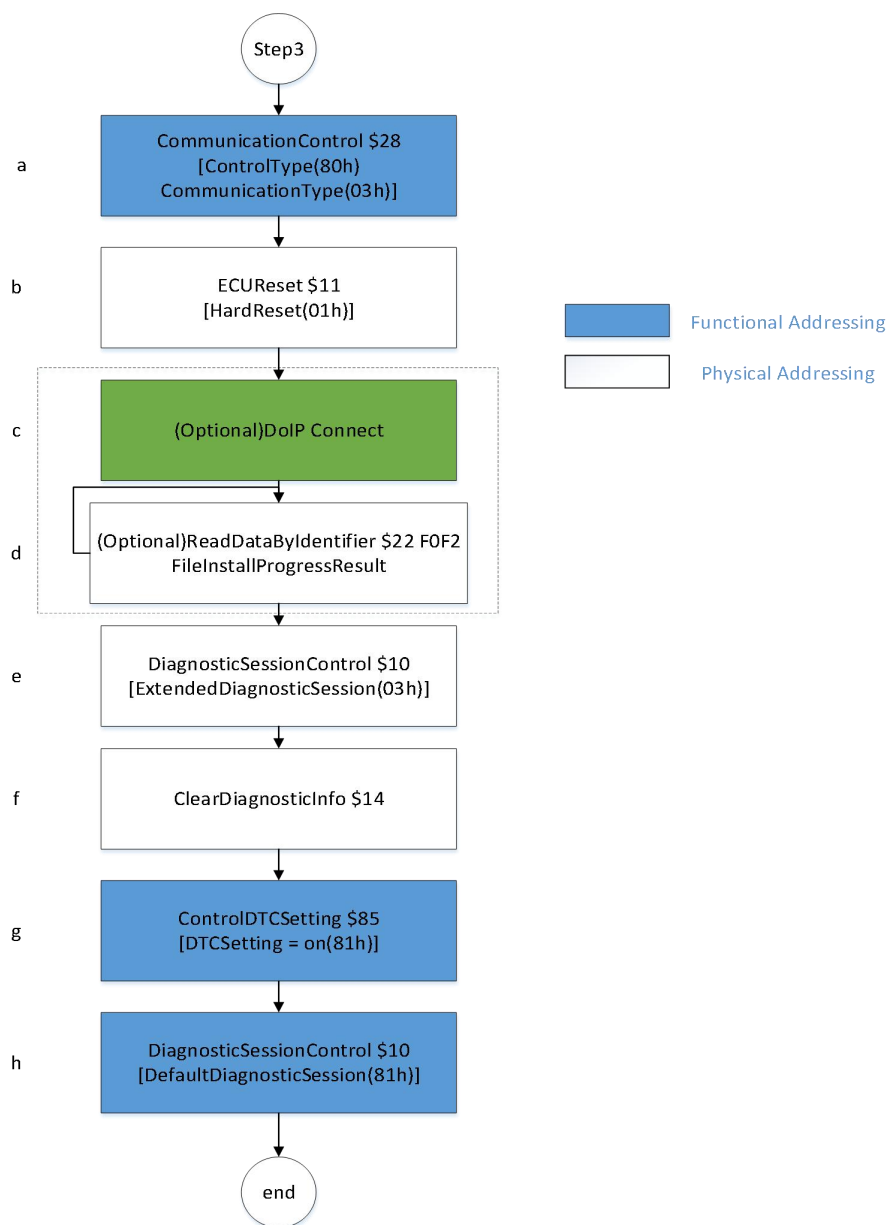


图8 后编程步骤

5.5 双备份控制器的要求

对于支持 AB 备份双分区运行的 ECU，其重编程规范流程跟一般 ECU 的重编程流程一样，同样分为预编程步骤，编程步骤，后编程步骤。其中预编程步骤和后编程步骤重编程时序是相同的。重编程流程的主要区别在于编程步骤的“读当前运行分区信息步骤”和“阻止自动切区”步骤可选支持。

双备份的 ECU 存在两个文件下载区域，分为“A 区”和“B 区”。A、B 区的刷写文件可以是不同的版本文件，也可以是相同的版本文件。双备份 ECU 属于硬件双分区的（也叫物理双分区），A、B 区的刷写文件可以是相同的版本文件，ECU 只需要发布一个软件包；双备份 ECU 属于软件双分区的（也叫逻辑双分区），A、B 区的刷写文件可以是不同的版本文件，ECU 需要发布 A/B 两个软件包。

AB 备份双分区运行的 ECU 须支持等待 Tester 发送切区和激活的指令，非双分区运行的 ECU 激活按照升级流程重启自激活。

双分区运行 ECU 刷写 A/B 运行区过程中，需支持控制 ECU 是否允许运行分区倒换（参考 5.5.1 章节）和触发分区倒换（参考 5.5.2 章节）。

双分区运行控制器刷写过程中未下发阻止自动切区时，运行分区倒换过程应如图 10 所示：

- 1) 运行区 A 刷写运行区 B，过程没有下发阻止自动切区命令（\$3101DD0F）；
- 2) 刷写完成并完成编程依赖性检查（对智能部件需完成版本安装）；
- 3) 重启进入 BootLoader；
- 4) BootLoader 引导系统启动进入运行分区 B；
- 5) 步骤 4）失败时需回退至 bootloader；
- 6) BootLoader 引导系统回滚至之前的运行分区 A；

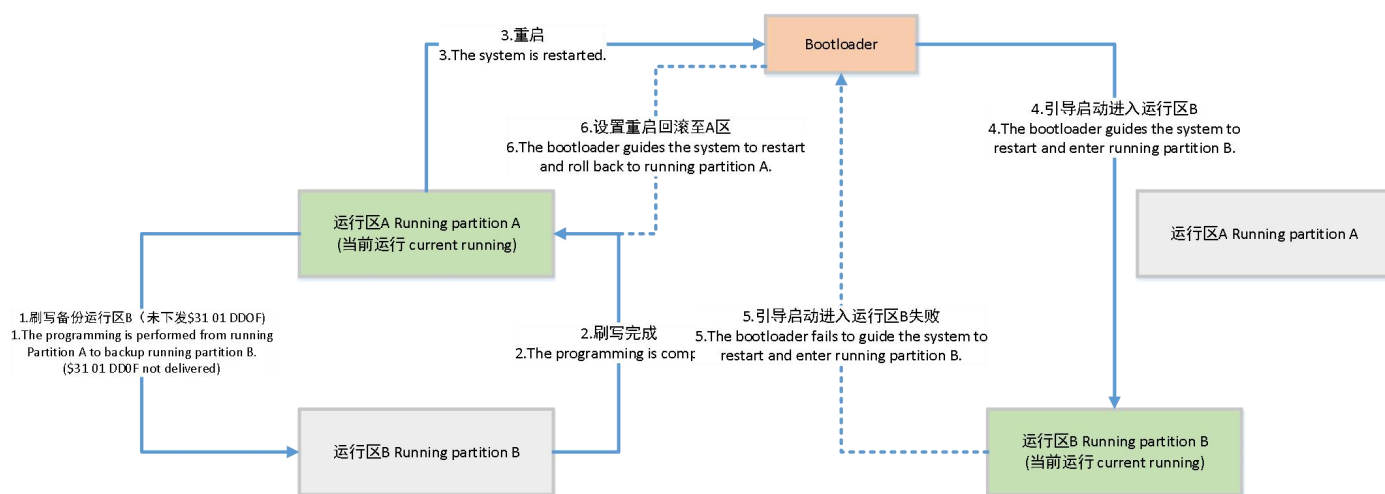


图 9 A/B 区刷写未发“禁止自动切区”后的分区倒换示意图

双分区运行控制器刷写过程中下发阻止自动切区时，运行分区倒换过程应如图 11 所示：

- 1) 运行区 A 刷写运行区 B，数据传输完成后下发阻止自动切区命令（\$3101DD0F）；
- 2)刷写完成并完成编程依赖性检查（对智能部件需完成版本安装）；
- 3)重启进入 BootLoader；
- 4)BootLoader 引导系统启动进入运行分区 A；
- 5)切区流程（\$3101DD04 命令，详细流程参考 5.6.2 章节）；
- 6)BootLoader 引导系统启动进入运行分区 B；
- 7)步骤 6）失败时需回退至 BootLoader；

8) BootLoader 引导系统回滚至之前的运行分区 A:

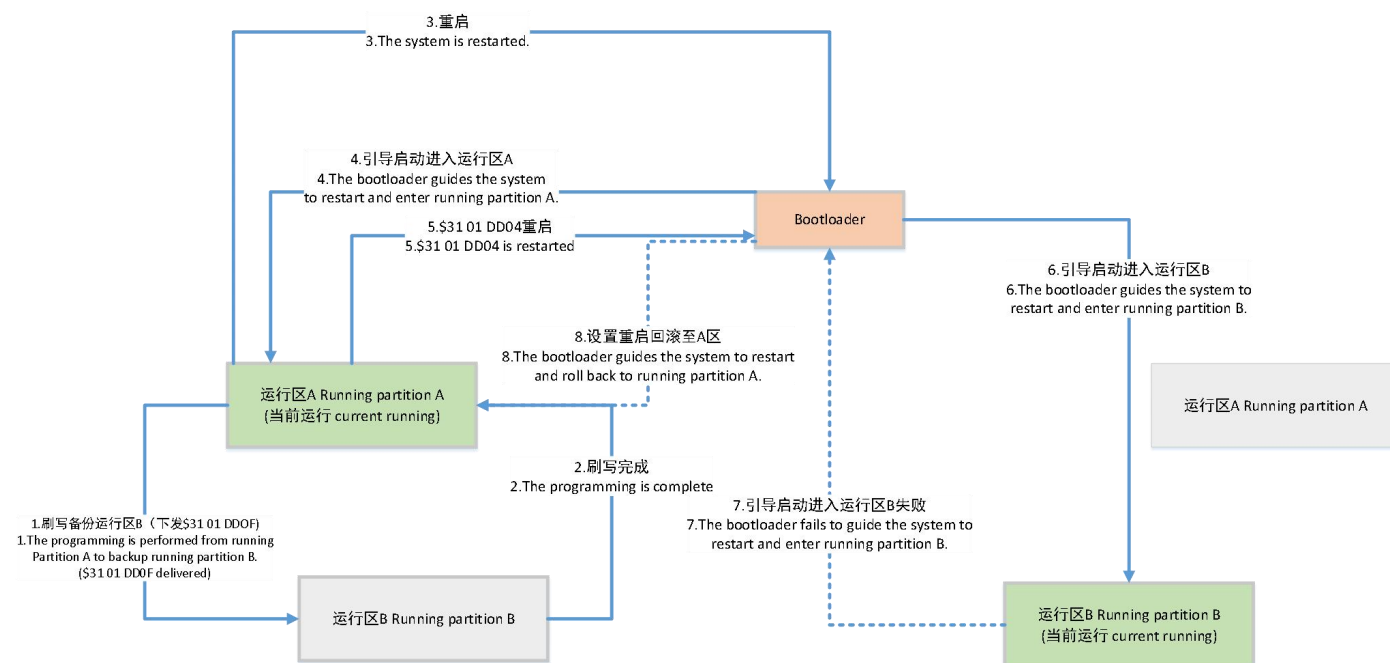


图 10 A/B 区刷写下发“禁止自动切区”&“切换运行区”后的分区倒换示意图

5.5.1 双分区运行 ECU 运行分区阻止切换例程

为了支持可灵活配置车辆刷写策略（如远程升级策略），双分区运行 ECU 在刷写文件或刷写程序完成下载后，需保证当前运行分区应用程序仍然有效和可靠，需发送阻止切区请求避免意外的运行分区自动切换动作，仅当再次接收到切区命令后（参考 5.5.2 章节）能够允许部件的运行分区倒换。

定义 `RoutineControlDataIdentifier-0xDD0F` 作为客户端请求服务端阻止自动切换运行分区的指令。

5.5.2 双分区运行 ECU 运行分区切换流程

双分区运行 ECU，应该支持运行分区切换流程，用于将程序运行分区从当前运行分区切换的另外的分区，在复位后生效。此流程所有消息均为物理寻址消息。比如当前是运行在 A 分区，经过此切换流程，ECU 复位后应该运行在 B 分区，反之亦然。如果切换失败，ECU 应该保持在切换前的可运行分区运行，即 A 分区切换到 B 分区切换失败时，最终运行分区是 A 分区软件版本。

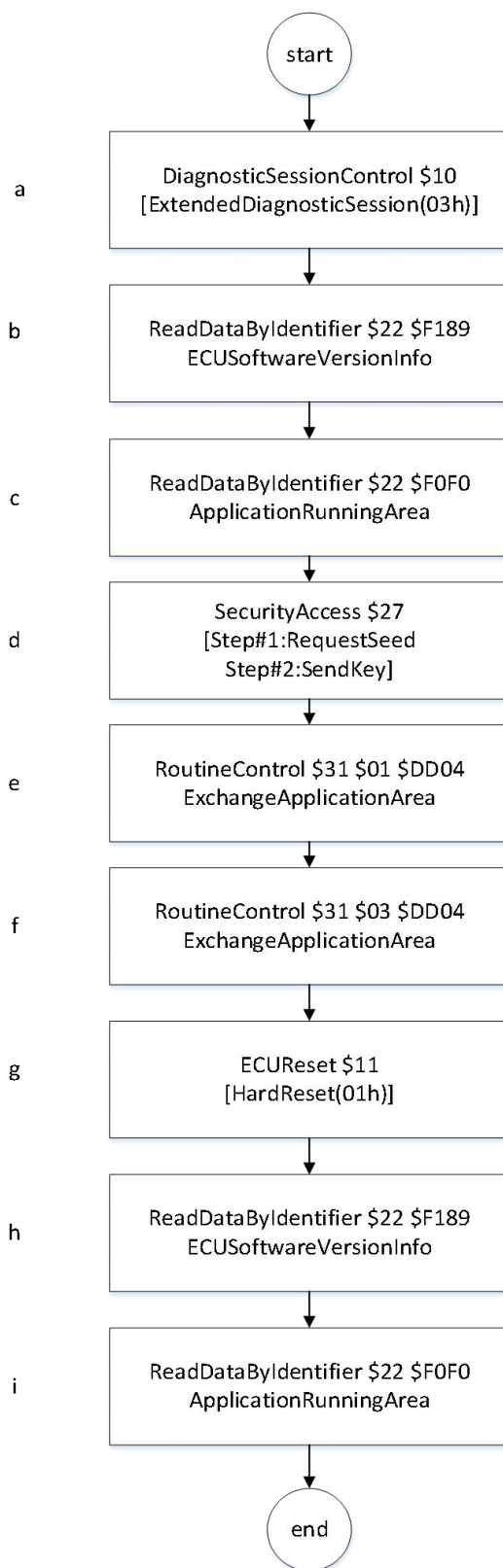


图 11 AB 备份双分区运行 ECU 分区切换流程图

- a) 诊断会话控制 10h03h: 发送\$1003 的物理寻址消息, 请求进入扩展会话模式。进入扩展会话模式后, Tester 需要发送周期性功能寻址的 TesterPresent (\$3E80) 服务, 以保持扩展诊断会话处于活动状态。发送间隔为 2.0 秒。
- b) 读取软件版本信息 22hF1h89h: 此步骤读取当前程序运行的分区的软件版本信息。
- c) 读取当前运行分区信息 22hF0hF0h: 此步骤用于读取当前程序运行的分区信息;
- d) 安全访问 27h: 例程控制需要进行安全访问解锁。此使用 2701/02 服务用于安全解锁。此安全解锁具体流程和算

法请参考《智能汽车 27 服务认证过程设计规范》文档。

e) 版本切换 31h01hDDh04h: ECU 在收到此例程消息后, 应该切换当前激活的分区, 并在复位后生效。版本切换流程耗时不能超过 30 分钟。

f) 查询版本切换结果 31h03hDDh04h: ECU 在版本切换过程中, 在接收到此例程消息后, 须按照当前版本切换实际进展进行响应。Tester 在发送 (f) 步骤后, 须以 1s 周期调用此服务, 查询版本切换结果。ECU 超时未响应此消息, Tester 不能当做错误处理, 应该继续查询, 直到获取到最终的结果状态。ECU 可能由于故障原因一直无法完成版本切换, 此步骤应该设置超时时间, 超时时间 T2=30 分钟没有完成或者结束, Tester 应该终止本次版本切换流程。

g) 复位 11h: 此服务将使 ECU 重启。ECU 在收到此消息复位前, 应该主动的关闭当前诊断链路的 Socket 通讯 (仅仅对 DoIP 的链路有此要求)。Tester 在发送完此命令后, 恢复和 ECU 的链路后, 再执行后续步骤。

(h) (i) 读取复位后的信息: 此两个步骤主要用于程序复位后, 读取复位后的程序软件版本信息和当前运行的分区信息用于判断此次版本切换流程的最终结果是否成功。如果在复位后读取到的程序分区信息以及软件版本信息和复位前一样, 则切换失败。具体信息请参考 (b) 步骤和 (c) 步骤。

5.6 诊断服务要求

表 2 定义了一个重编程 ECU 的 Bootloader 软件需要的最小诊断服务。为了满足执行 ECU 非易失性内存的编程, 表格中的服务必须要支持。

表 2 预编程步骤支持的诊断服务

服务	子功能/数据参数	寻址信息		应用模式		Bootloader模式		
		物理 Phy	功能 Fun	默认 Def	扩展 Ext	默认 Def	扩展 Ext	编程 Pro
诊断会话控制 (10h)	会话类型=扩展诊断会话 (03h)	√ ^[1]	√	√	√	√	√	--
例程控制 (31h)	例程控制类型=开始例程 (01h) DID=检查预编程条件 (02h03h)	√	--	--	√	--	√	--
通信控制 (28h)	控制类型=不能接收和发送 (03h) Controltype=disablereceiveands end 通信类型=网络管理报文和常 规应用报文 (03h)	--	√	--	√	--	√	--
控制DTC设置 (85h)	DTC设置类型=关 (02h)	--	√	--	√	--	√	--
通过DID读数据 (22h)	DID= (供应商规定)	√	√	√	√	√	√	--
诊断设备在线	Sub-function=80h	√	√	√	√	√	√	√
注[1]: √-需要支持的服务								

表 3 主编程步骤支持的诊断服务

服务	子功能/数据参数	寻址信息		应用模式		Bootloader模式		
		物理 Phy	功能 Fun	默认 Def	扩展 Ext	默认 Def	扩展 Ext	编程 Pro
诊断会话控制(10h)	会话类型=编程诊断会话 (02h)	√	--	--	√	--	√	√
安全访问(27h)	安全访问类型=请求种子 (11h) 发送密钥 (12h)	√	--	--	--	--	--	√
通过DID写数据 (2Eh)	DID=F1h84h	√	--	--	--	--	--	√
请求下载(34h/38h)	--	√	--	--	--	--	--	√
数据传输(36h)	--	√	--	--	--	--	--	√

请求传输退出(37h)	--	√	--	--	--	--	--	√
例程控制(31h)	例程控制类型=开始例程 (01h) DID=擦除内存 (FFh00h)	√	--	--	--	--	--	√
例程控制(31h)	例程控制类型=开始例程 (01h) DID=检查编程完整性 (02h02h)	√	--	--	--	--	--	√
例程控制(31h)	例程控制类型=开始例程 (01h) DID=安全签名校验 (FFh01h)	√	--	--	--	--	--	√
例程控制(31h)	例程控制类型=开始例程 (01h) DID=启动安装 (FDh08h)	√	--	--	--	--	--	√
ECU复位 (11h)	复位类型=硬复位 (01h)	√	--	--	--	√	√	√
诊断在线(3Eh)	子功能=80h	√	√	√	√	√	√	√

表 4 后编程步骤支持的服务

服务	子功能/数据参数	寻址信息		应用模式		Bootloader模式		
		物理 Phy	功能 Fun	默认 Def	扩展 Ext	默认 Def	扩展 Ext	编程 Pro
ECU复位 (11h)	复位类型=硬复位 (01h)	√	--	--	--	√	√	√
诊断会话控制 (10h)	会话类型=默认会话模式(01h)	--	√	--	√	√	√	√
清除诊断信息 (14h)	DTC组=FFFFFF	--	√	√	--	--	--	--

5.5.1 2E 服务

数据标识符 F1h84h-“指纹”，此数据标识符应该可读可写（R/W），其遵循表 4 所示的格式：

表 4 写入数据-“指纹”请求

Byte	Parametername	Value (hex)
#0	WriteDataByIdentifierRequestServiceID	2E
#1	FingerprintDID (HByte)	F1
#2	FingerprintDID (Lbyte)	84
#3	programmingDateYY (byte0,Hexcoded)	00-63
#4	programmingDateMM (byte1,Hexcoded)	00-0C
#5	programmingDateDD (byte2,Hexcoded)	00-1F
#6~#11	TesterSerialNumber	00-FF

5.5.2 22 服务

数据标识符 F1h84h“指纹”，通过此服务可以追踪 ECU 程序的指纹信息，其遵循表 5 所示的格式：

表 5 22 服务-“读指纹”肯定响应

Byte	Parametername	Value(hex)
#0	ReadDataByIdentifierResponseServiceID	62
#1	FingerprintDID(HByte)	F1
#2	FingerprintDID(LByte)	84h
#3	ProgrammingDateYear(Hbyte0,Hexcoded)	00-63
#4	ProgrammingDateMonth(Mbyte0,Hexcoded)	00-0C

#5	ProgrammingDateDate(Lbyte0,Hexcoded)	00-1F
#6~#11	TesterSerialNumber	00-FF

在数据传输前，强制将“指纹”写到 ECU 内存中。“指纹”标识了哪个诊断仪什么时间对 ECU 内存做了修改。ECU 需要维护一个尝试编程次数的计数器(DID=0xF0F1)，其遵循表 6 所示的格式：

表 6 22 服务-“尝试编程计数器”格式定义

尝试编程次数 0xF0F1	类型：无符号 32 位整型	00-FFh	尝试编程次数 Byte#1（MSB）
		00-FFh	尝试编程次数 Byte#2
		00-FFh	尝试编程次数 Byte#3
		00-FFh	尝试编程次数 Byte#4（LSB）

数据标识符 F0hF0h“当前运行分区信息”，通过此服务可以读取双分区运行 ECU 当前运行分区信，其遵循表 7 所示的格式：

表 7 22 服务-“读当前运行分区信息”肯定响应

Byte	Value(hex)	Description
#0	62	服务ID
#1	F0	0xF0F0 当前运行分区信息
#2	F0h	
#3	41/42h	“A/B”

数据标识符 F0hF2h“软件安装结果”，通过此服务可以获取 ECU 装进度和结果，智能部件必须支持此数据标识符。其遵循表 8 所示的格式，在编程序列的安装进度获取中，FileInstallStatus 字段应该只允许响应 00h/01h/02h/04h，不允许响应 03h/05h。

表 8 22 服务-“软件安装结果”肯定响应

Byte	Value(hex)	Description
#0	62	服务ID
#1	F0	0xF0F2 软件安装结果
#2	F2h	
#3	00-FFh	FileInstallStatus 00：成功 01：其他失败 02：安装中 03：校验中（激活中） 04：安装失败 05：校验失败（激活失败）
#4	00-64h	安装进度（或校验激活进度）百分比（%）如 63h=99%

ECU 进行编程依赖关系检查步骤，内部维护一个编程依赖检查成功的计数器(DID=0xF0F3)，并保存在 Flash 中，每次编程依赖检查成功后此计数器加 1，当达到最大允许编程次数（最大允许编程次数设置为 0 时，不限制编程次数，默认为 0），ECU 应该禁止编程，编程依赖检查返回 NRC22 的否定响应消息。其遵循表 9 所示的格式：

表 9 22 服务-“编程依赖检查成功计数器”格式定义

编程依赖检查成功计数器 0xF0F3	类型：无符号 32 位整型	00-FFh	编程依赖检查成功计数器 Byte#1（MSB）
		00-FFh	编程依赖检查成功计数器 Byte#2
		00-FFh	编程依赖检查成功计数器 Byte#3
		00-FFh	编程依赖检查成功计数器 Byte#4（LSB）

5.5.3 例程控制-安全签名校验

安全签名校验的请求和响应报文需要遵循表 7 和表 8 所示的格式：

表 7 例程控制-“安全签名校验”请求

Byte	Value（hex）	Description
#0	31	服务ID
#1	01	启动
#2-#3	0202	0x0202安全签名验证
#4	00-FFh	安全签名值
...	00-FFh	
#N	00-FFh	

表 8 例程控制-“安全签名校验”肯定响应

Byte	Value（hex）	Description
#0	71	服务ID
#1	01	启动检查
#2-#3	0202	0x0202安全签名验证
#4	00/01h	00：成功； 01：供应商签名校验失败； 02：主机厂签名校验失败； 03：密钥不存在失败

5.5.4 例程控制-检查预编程条件

检查预编程条件的请求和响应报文需要遵循表 9 和表 10 所示的格式：

表 9 例程控制-“检查预编程条件”请求

Byte	Parametername	Value（hex）
#0	RoutineControlRequestServiceId	31
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier checkProgrammingPreconditions	0203

表 10 例程控制-“检查预编程条件”肯定响应

Byte	Parametername	Value（hex）
#0	RoutineControlResponseServiceId	71
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier	0203

	checkProgrammingPreconditions	
--	-------------------------------	--

5.5.5 例程控制-擦除内存

擦除内容的请求和响应报文需要遵循表 9 和表 10 所示的格式：

表 11 例程控制-“擦除内存”请求

Byte	Parametername	Value (hex)
#0	RoutineControlRequestServiceId	31
#1	routineControlType startRoutine	01
#2-#3	RoutineIdentifier eraseMemory	FF00
#4-#n7	memoryAddress4byteseraseaddress	00-FF
#8-#11	memorySize4byteserasesize	00-FF

表 12 例程控制-“擦除内存”肯定响应

Byte	Parametername	Value (hex)
#0	RoutineControlResponseServiceId	71
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier eraseMemory	FF00

5.5.6 例程控制-检查编程依赖性

检查编程依赖性的请求和响应报文需要遵循表 13 和表 14 所示的格式：

表 13 例程控制-“检查编程依赖性”请求

Byte	Parametername	Value (hex)
#0	RoutineControlRequestServiceId	31
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier checkProgrammingDependencies	FF01

表 14 例程控制-“检查编程依赖性”肯定响应

Byte	Parametername	Value (hex)
#0	RoutineControlResponseServiceId	71
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier checkProgrammingDependencies	FF01

5.5.7 例程控制-启动安装

主编程步骤中，当更新文件下载完成后，诊断仪会通过例程控制服务控制被刷写 ECU 进行安装，被刷写节点应先响应诊断请求，然后执行安装。其遵循表 15、16 所示的格式：

表 15 例程控制-“启动安装”请求

Byte	Parametername	Value (hex)
------	---------------	-------------

#0	RoutineControlRequestServiceId	31
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier StartInstallation	FD08

表 16 例程控制-“启动安装”肯定响应

Byte	Parametername	Value (hex)
#0	RoutineControlResponseServiceId	71
#1	routineControlType startRoutine	01
#2-#3	routineIdentifier StartInstallation	FD08

5.5.8 例程控制-阻止自动切区

此步骤用于限制支持 A/B 区 ECU 做自动切区激活的动作，支持 A/B 区运行的部件必须支持此步骤（如无法支持需要向 OEM 申请认可），或非 A/B 区运行部件可不支持此步骤。支持 A/B 区的传统 ECU 在刷写程序完成传输后可下发该指令阻止自动切区的动作，防止发生非预期的自动切区动作。其遵循表 17、18 所示的格式：

表 17 例程控制-“阻止自动切区”请求

Byte	Value (hex)	Description
#0	31	服务ID
#1	01	启动
#2-#3	DD0F	0xDD0F 阻止自动切区动作

表 18 例程控制-“阻止自动切区”肯定响应

Byte	Value (hex)	Description
#0	71	服务ID
#1	01	启动检查
#2-#3	DD0F	0xDD0F 阻止切区动作
#4	00/01h	00: 成功 01: 失败（需要在诊断问卷中注明可能失败的原因）

5.5.9 例程控制-版本切换

ECU 在收到此例程消息后，应该切换当前激活的分区，并在复位后生效。版本切换流程耗时不能超过 30 分钟。其遵循表 19、20 所示的格式：

表 19 例程控制-“版本切换”请求

Byte	Value (hex)	Description
#0	31	服务ID
#1	01	启动
#2-#3	DD04	0xDD04 版本切换

表 20 例程控制-“版本切换”肯定响应

Byte	Value (hex)	Description
#0	71	服务ID
#1	01	启动检查

#2-#3	DD04	0xDD04 版本切换
#4	00-02h	00: 成功 01: 失败（需要在诊断问卷中注明可能失败的原因）

5.5.10 例程控制-查询版本切换结果

ECU 在版本切换过程中，在接收到此例程消息后，须按照当前版本切换实际进展进行响应。其遵循表 21、22 所示的格式：

表 21 例程控制-“查询版本切换结果”请求

Byte	Value (hex)	Description
#0	31	服务ID
#1	03	查询例程结果
#2-#3	DD04	0xDD04 版本切换

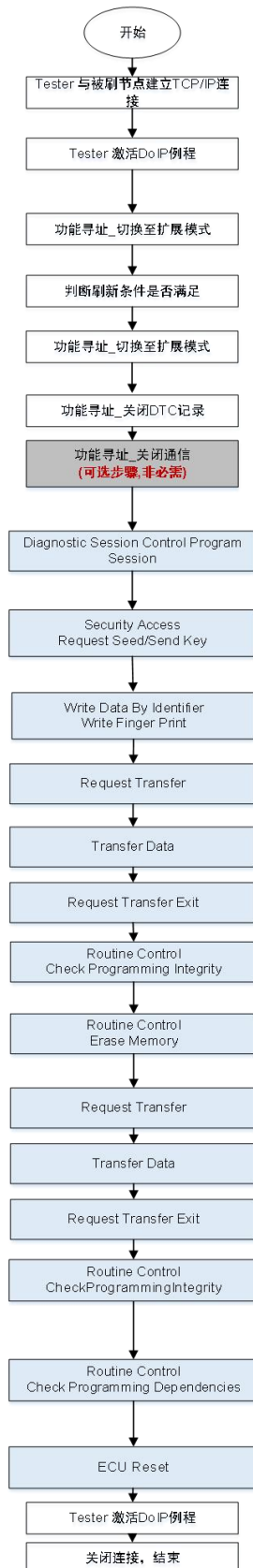
表 22 例程控制-“查询版本切换结果”肯定响应

Byte	Value (hex)	Description
#0	71	服务ID
#1	03	查询例程结果
#2-#3	DD04	0xDD04 版本切换
#4	00-02h	00: 切区完成 01: 切区失败 02: 切区中

附录 A
(规范性)
I 类刷新流程

BootloaderOnIP补充说明

—— I 类刷新流程数据流范例



示例用逻辑地址:

Tester: 0x0E80

ECU: 0x078E

Tester 请求: 02 FD 00 05 00 00 00 08 0E 80 00 00 00 00 00 00 00
ECU节点响应: 02 FD 00 06 00 00 00 0D 0E 80 07 8E 10 00 00 00 00 00 00 00

Tester 请求: 02 FD 80 01 00 00 00 06 0E 80 E4 00 10 03
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 10
ECU节点响应: 02 FD 80 01 00 00 00 0A 07 8E 0E 80 50 03 XX XX XX XX, xx为时间参数

Tester 请求: 02 FD 80 01 00 00 00 08 0E 80 07 8E 31 01 02 03
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 31
ECU节点响应: 02 FD 80 01 00 00 00 08 07 8E 0E 80 71 01 02 03

Tester 请求: 02 FD 80 01 00 00 00 06 0E 80 E4 00 10 83
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 10

Tester 请求: 02 FD 80 01 00 00 00 06 0E 80 E4 00 85 82
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 85

Tester 请求: 02 FD 80 01 00 00 00 07 0E 80 E4 00 28 83 03
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 28

Tester 请求: 02 FD 80 01 00 00 00 06 0E 80 07 8E 10 02
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 10
ECU节点响应: 02 FD 80 01 00 00 00 0A 07 8E 0E 80 50 02 XX XX XX XX, xx为时间参数

Tester 请求: 02 FD 80 01 00 00 00 06 0E 80 07 8E 27 11
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 27
ECU节点响应: 02 FD 80 01 00 00 00 0A 07 8E 0E 80 67 11 XX XX XX XX, xx为随机数种子
Tester 请求: 02 FD 80 01 00 00 00 0A 0E 80 07 8E 27 12 XX XX XX XX, xx为密钥
ECU节点响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 27
ECU节点响应: 02 FD 80 01 00 00 00 06 07 8E 0E 80 67 12

Tester 请求: 02 FD 80 01 00 00 00 10 0E 80 07 8E 2F F1 84 (9Bytes 指纹信息)
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 2E
ECU节点响应: 02 FD 80 01 00 00 00 07 07 8E 0E 80 6E F1 84

Tester 请求: 02 FD 80 01 00 00 00 0F 0E 80 07 8E 34 00 44 4bytes地址+ 4Bytes大小长度
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 34
ECU节点响应: 02 FD 80 01 00 00 00 08 07 8E 0E 80 74 20 XX XX (根据软件包大小自定义)

Tester 请求: 02 FD 80 01 00 00 00 XX XX 0E 80 07 8E 36 01 XX XX ... XX XX = XX XX + 4
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 36
ECU节点响应: 02 FD 80 01 00 00 00 06 07 8E 0E 80 76 01

Tester 请求: 02 FD 80 01 00 00 00 05 0E 80 07 8E 37
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 37
ECU节点响应: 02 FD 80 01 00 00 00 05 07 8E 0E 80 77

Tester 请求: 02 FD 80 01 00 00 XX XX 0E 80 07 8E 31 01 02 02 XX XX ... (根据签名值长度定义)
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 31
ECU节点响应: 02 FD 80 01 00 00 00 09 07 8E 0E 80 71 01 02 02 00

Tester 请求: 02 FD 80 01 00 00 00 10 0E 80 07 8E 31 01 FF 00 4bytes地址+ 4Bytes大小长度
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 31
ECU节点响应: 02 FD 80 01 00 00 00 08 07 8E 0E 80 71 01 FF 00

Tester 请求: 02 FD 80 01 00 00 00 0F 0E 80 07 8E 34 00 44 4bytes地址+ 4Bytes大小长度
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 34
ECU节点响应: 02 FD 80 01 00 00 00 08 07 8E 0E 80 74 20 XX XX (根据软件包大小自定义)

Tester 请求: 02 FD 80 01 00 00 00 XX XX 0E 80 07 8E 36 01 XX XX ... XX XX = XX XX + 4
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 36
ECU节点响应: 02 FD 80 01 00 00 00 06 07 8E 0E 80 76 01

Tester 请求: 02 FD 80 01 00 00 00 05 0E 80 07 8E 37
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 37
ECU节点响应: 02 FD 80 01 00 00 00 05 07 8E 0E 80 77

Tester 请求: 02 FD 80 01 00 00 XX XX 0E 80 07 8E 31 01 02 02 XX XX ... (根据签名值长度定义)
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 31
ECU节点响应: 02 FD 80 01 00 00 00 09 07 8E 0E 80 71 01 02 02 00

Tester 请求: 02 FD 80 01 00 00 00 08 0E 80 07 8E 31 01 FF 01
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 31
ECU节点响应: 02 FD 80 01 00 00 00 08 07 8E 0E 80 71 01 FF 01

Tester 请求: 02 FD 80 01 00 00 00 06 0E 80 07 8E 11 01
ECU的ACK响应: 02 FD 80 02 00 00 00 06 07 8E 0E 80 00 11
ECU节点响应: 02 FD 80 01 00 00 00 0A 07 8E 0E 80 51 01
Tester 请求: 02 FD 00 05 00 00 00 08 0E 80 00 00 00 00 00 00 00 00
ECU节点响应: 02 FD 00 06 00 00 00 0D 0E 80 07 8E 10 00 00 00 00 00 00 00 00

附录 B
(规范性)

双分区运行 ECU 禁止自动切区和恢复自动切区实现参考方案

本附录为控制器的参考实现方案，目的是方便开发者理解智能汽车的双分区运行刷写规范的需求，控制器的具体实现方案本规范不做强制要求。图 13 是双分区运行 ECU 的启动分区标识位和禁止切区状态位的更新机制和重启的参考方案时序图。

本参考方案的三个设计前提说明：

- 1) 本规范中所述自动切区动作，定义为 ECU 在某个条件满足后自动更新启动分区标志位。如编程依赖性检查成功后，ECU 即可认为刷写成功即可更新启动分区标志位，此时自动切区动作包含在编程依赖性检查步骤中。
- 2) ECU 应维护一个禁止自动切区的状态 Boolean 值 FrbExchgAreaFlag。当 FrbExchgAreaFlag=False 时，表示 ECU 未开启阻止自动切区；当 FrbExchgAreaFlag=True 时，表示 ECU 开启阻止自动切区。状态位 FrbExchgAreaFlag 的默认值为 False 且允许掉电丢失。
- 3) ECU 应维护一个启动分区标识位 StrtRngAreaFlag。当 StrtRngAreaFlag=A，表示下次重启 ECU 需要在 A 分区运行；当 StrtRngAreaFlag=B，表示下次重启 ECU 需要在 B 分区运行。启动分区标识位应存储在非易失性存储区，其值不允许掉电丢失。

该方案下的三个场景实现说明：

- 1) 场景一：接收禁止自动切区命令场景。当 ECU 接收到禁止切区指令并能够成功响应后，FrbExchgAreaFlag 须置为 True，否则保持为 False。

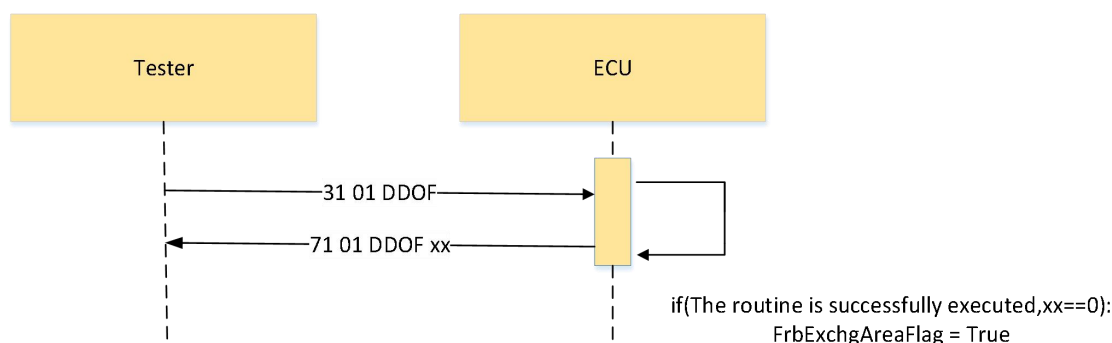


图 12 双运行分区 ECU 禁止切区状态值赋值场景

- 2) 场景二：接收编程依赖性检查命令场景，该场景下，如设计前提 1)所述自动切区动作包含在编程依赖性检查步骤中。当 ECU 接收到编程依赖性检查并能够成功响应，须依赖禁止切区状态值 FrbExchgAreaFlag 判定是否需要执行自动更新启动标识位 StrtRngAreaFlag 的动作。当 FrbExchgAreaFlag=True 时，须保持 StrtRngAreaFlag 当前的值，并重置 FrbExchgAreaFlag=False；当 FrbExchgAreaFlag=False 时，须将 StrtRngAreaFlag 值置为刷新区。

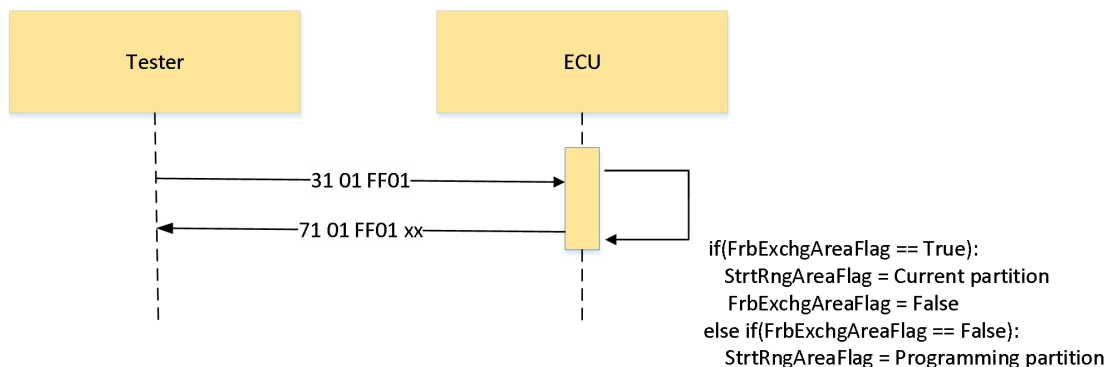


图 13 双运行分区 ECU 禁止自动切区场景-自动切区动作在编程依赖性检查步骤中

- 3) 场景三：接收切区命令场景。该场景下强制切区不依赖于禁止切区状态值 FrbExchgAreaFlag 的判定，可以强制

切换启动分区标识位为对区。如果 ECU 能够成功响应，则需将启动分区标识 StrtRngAreaFlag 更新至对分区，否则该标识位须保持为当前运行分区。

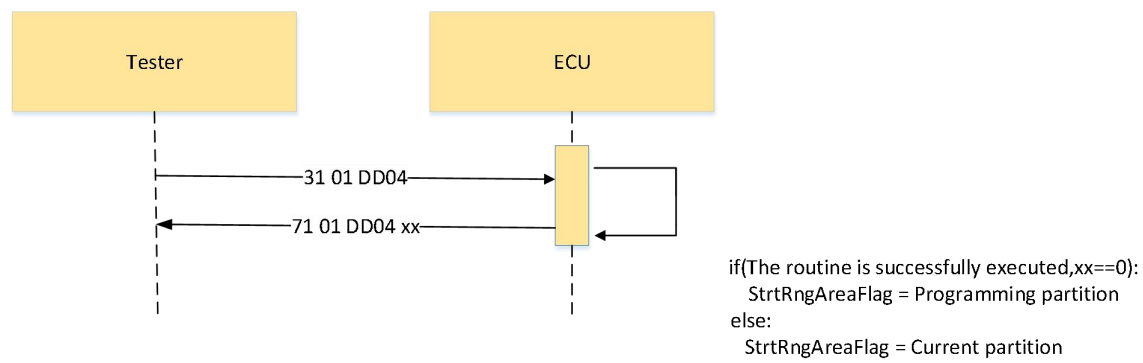


图 14 双备份 ECU 接收切区指令场景